

SQL cheat sheet

Table of contents

1. SELECT	2. INSERT	3. UPDATE
4. DELETE	5. CREATE	6. DROP
7. ALTER	8. TRUNCATE	9. PRIMARY KEY
10. FOREIGN KEY	11. UNIQUE	12. NOT NULL
13. CHECK	14. DEFAULT	15. INNER JOIN
16. LEFT JOIN	17. RIGHT JOIN	18. FULL OUTER JOIN
19. CROSS JOIN	20. SELF JOIN	21. UNION
22. COUNT()	23. SUM()	24. AVG()
25. MIN()	26. MAX()	27. GROUP BY
28. HAVING	29. COALESCE	30. Subqueries and nested queries
31. Indexes	32. Stored procedures and functions	33. Views
34. Transactions and concurrency control	35. Normalization and database design	36. First Normal Form (1NF)
37. Second Normal Form (2NF)	38. Third Normal Form (3NF)	39. Boyce-Codd Normal Form (BCNF)
40. Denormalization	41. SQL performance optimization	42. Common Table Expressions (CTE) and recursive queries
43. Window functions	44. JSON and XML handling	45. CASE
46. Operators	47. LIKE	48. IN
49. Aliases	50. Triggers	51. N+1 problem
52. Database Metadata / System Information		

SELECT

```
SELECT *  
FROM employees;
```

This fetches all rows and columns from the employees table.

```
SELECT first_name, last_name, salary  
FROM employees;
```

Instead of selecting all columns, you can specify which ones you want. This retrieves only first_name, last_name, and salary from the employees table.

```
SELECT *  
FROM employees  
WHERE department = 'IT';
```

The WHERE clause filters records based on conditions. This retrieves all employees in the IT department.

```
SELECT first_name, salary  
FROM employees  
ORDER BY salary DESC;
```

ORDER BY sorts the output in ascending (ASC) or descending (DESC) order. This sorts employees by salary in descending order.

```
SELECT *  
FROM employees  
LIMIT 5;
```

Limits the number of rows returned. This returns only the first five employees.

```
SELECT DISTINCT department  
FROM employees;
```

Retrieves unique values from a column. This lists all unique department names.

```
SELECT department, COUNT(*)  
FROM employees  
GROUP BY department;
```

Groups rows (aggregating data) that have the same values in specified columns. This counts the number of employees in each department.

```
SELECT department, COUNT(*)
FROM employees
GROUP BY department
HAVING COUNT(*) > 10;
```

Filters results after GROUP BY using conditions. This returns departments that have more than 10 employees.

```
SELECT employees.first_name, departments.department_name
FROM employees
JOIN departments ON employees.department_id = departments.id;
```

Retrieves data from multiple related tables. This combines employees and departments tables to show each employee's department.

```
SELECT first_name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

A query inside another query. This finds employees earning more than the average salary.

```
SELECT first_name, salary,
CASE
    WHEN salary > 50000 THEN 'High Salary'
    ELSE 'Low Salary'
END AS salary_category
FROM employees;
```

Adds conditional logic within a query. This labels employees based on their salary.

INSERT

```
INSERT INTO employees (first_name, last_name, department, salary)
VALUES ('John', 'Doe', 'IT', 60000);
```

You can insert a new row by specifying values for each column. This inserts a new employee with the given values.

```
INSERT INTO employees (first_name, last_name, department, salary)
VALUES
```

```
  ('Alice', 'Smith', 'HR', 50000),
  ('Bob', 'Johnson', 'Finance', 55000),
  ('Charlie', 'Brown', 'IT', 62000);
```

You can insert multiple rows at once by separating values with commas. This adds three employees in a single query.

```
INSERT INTO employees
VALUES (101, 'Emma', 'Wilson', 'Marketing', 70000);
```

If you're inserting values for all columns, you can omit the column names, but the values must match the table's column order. This is risky because if the table structure changes, the query might break.

```
INSERT INTO employees_backup (first_name, last_name, department, salary)
SELECT first_name, last_name, department, salary FROM employees;
```

You can copy data from one table to another. This copies all employee data into the employees_backup table.

```
INSERT INTO employees (first_name, last_name, department)
VALUES ('David', 'Miller', 'Sales');
```

If a column has a DEFAULT value, you can skip it. In this example if salary has a default value (e.g., 40000), it will be used automatically.

```
INSERT INTO orders (customer_name, total_amount, created_at)
VALUES ('John Doe', 150.75, NOW());
```

If a table has a created_at column, you can insert the current timestamp. In this example NOW() gets the current date and time.

```
INSERT INTO employees (first_name, last_name, department, salary)
VALUES ('Sophia', 'Anderson', NULL, 52000);
```

If a column allows NULL, you can insert a NULL value. In this example the department value will be NULL.

```
INSERT INTO employees (first_name, last_name, department, salary)
VALUES ('Michael', 'Clark', 'IT', 58000);
```

If a table has an auto-increment primary key, you don't need to insert a value for it. In this example the database will generate the ID automatically.

UPDATE

```
UPDATE employees
SET salary = 65000
WHERE employee_id = 101;
```

You can update one or more columns of a specific row using the WHERE clause. This updates the salary of the employee with employee_id = 101.

```
UPDATE employees
SET salary = 70000, department = 'Finance'
WHERE employee_id = 102;
```

You can update multiple columns at once by separating them with commas. This updates both the salary and department of employee 102.

```
UPDATE employees
SET salary = salary + 5000
WHERE department = 'IT';
```

If the WHERE clause matches multiple rows, all of them will be updated. This increases the salary of all employees in the IT department by 5000.

```
UPDATE employees
SET salary = 50000;
```

Be careful when using UPDATE without a WHERE clause—it will update all rows. This sets the salary of every employee to 50000 (which is usually not intended).

```
UPDATE employees
SET salary =
CASE
    WHEN department = 'IT' THEN salary + 7000
    WHEN department = 'HR' THEN salary + 5000
    ELSE salary + 3000
END;
```

You can use CASE to apply different updates based on conditions. This increases salaries based on department.

```
UPDATE employees
```

```
SET salary = (SELECT AVG(salary) FROM employees)
WHERE department = 'Sales';
```

You can update values using data from another table. This sets the salary of all Sales employees to the average salary of all employees.

```
UPDATE employees
JOIN departments ON employees.department_id = departments.id
SET employees.salary = employees.salary * 1.1
WHERE departments.department_name = 'Marketing';
```

You can update a table based on data from another table. This increases the salary of employees in the Marketing department by 10%.

```
UPDATE employees
SET salary = 60000
WHERE department = 'HR'
LIMIT 5;
```

Some databases support LIMIT in UPDATE statements to restrict updates. This works in MySQL, but not in some other databases like SQL Server or PostgreSQL.

```
START TRANSACTION;

UPDATE employees
SET salary = 80000
WHERE department = 'Finance';

ROLLBACK; -- This undoes the update

COMMIT; -- This saves the update permanently
```

If you want to ensure safe updates, use transactions. If you ROLLBACK, the update is canceled. If you COMMIT, the update is saved.

DELETE

```
DELETE FROM employees
WHERE employee_id = 101;
```

To delete specific records, use the WHERE clause. This deletes the employee with employee_id = 101.

```
DELETE FROM employees
WHERE department = 'HR';
```

If the WHERE clause matches multiple rows, all of them will be deleted. This removes all employees in the HR department.

```
DELETE FROM employees;
```

If you want to remove all rows from a table, use DELETE without a WHERE clause. Be careful! This deletes all records but keeps the table structure and does not reset auto-increment counters. If you need to reset auto-increment counters, use truncate.

```
DELETE employees
FROM employees
JOIN departments ON employees.department_id = departments.id
WHERE departments.department_name = 'Sales';
```

You can delete records based on related tables using JOIN. This deletes all employees who work in the Sales department.

```
DELETE FROM employees
WHERE department_id IN (SELECT id FROM departments WHERE department_name = 'HR');
```

You can delete records based on another table's data. This removes employees working in the HR department.

```
DELETE FROM employees
WHERE department = 'IT'
LIMIT 5;
```

Some databases (like MySQL) allow you to limit the number of deleted rows. This deletes only 5 employees from the IT department.

```
START TRANSACTION;

DELETE FROM employees WHERE department = 'Finance';

ROLLBACK; -- Cancels the deletion
COMMIT; -- Saves the deletion permanently
```

To prevent accidental deletions, use transactions. If you ROLLBACK, the deleted records are restored. If you COMMIT, the deletion is permanent.

CREATE

```
CREATE DATABASE company_db;
```

You can create a new database like this.

```
USE company_db;
```

You can use the newly created database like this.

```
CREATE TABLE employees (  
  employee_id INT PRIMARY KEY AUTO_INCREMENT,  
  first_name VARCHAR(50) NOT NULL,  
  last_name VARCHAR(50) NOT NULL,  
  department VARCHAR(50),  
  salary DECIMAL(10,2),  
  hire_date DATE DEFAULT CURRENT_DATE  
);
```

To create a new table, specify column names and data types.

- employee_id INT PRIMARY KEY AUTO_INCREMENT - Unique ID that auto-increments.
- VARCHAR(50) NOT NULL - Stores text with a max of 50 characters, required.
- DECIMAL(10,2) - Stores salary with up to 10 digits and 2 decimal places.
- DEFAULT CURRENT_DATE - Sets the default hire date as today.

```
CREATE TABLE departments (  
  department_id INT PRIMARY KEY AUTO_INCREMENT,  
  department_name VARCHAR(100) UNIQUE NOT NULL,  
  manager_id INT,  
  FOREIGN KEY (manager_id) REFERENCES employees(employee_id) ON DELETE SET NULL  
);
```

Constraints ensure data integrity.

- PRIMARY KEY - Ensures unique records.
- UNIQUE - Ensures values in a column are unique.
- FOREIGN KEY - Links to another table (employees), setting manager_id to NULL if the referenced record is deleted.


```
CREATE TABLE employees_backup AS  
SELECT * FROM employees;
```

You can create a new table based on an existing one. This copies all data and structure from employees to employees_backup.

```
CREATE INDEX idx_department ON employees(department);
```

Indexes speed up queries. This creates an index on the department column for faster searches.

```
CREATE VIEW high_salary_employees AS  
SELECT first_name, last_name, salary FROM employees WHERE salary > 70000;
```

```
-- You can now query the view like a table:
```

```
SELECT * FROM high_salary_employees;
```

A VIEW is a virtual table based on a query.

```
CREATE PROCEDURE GetEmployeesByDepartment(IN dept_name VARCHAR(50))  
BEGIN  
    SELECT * FROM employees WHERE department = dept_name;  
END;
```

```
-- Execute the procedure:
```

```
CALL GetEmployeesByDepartment('IT');
```

Stored procedures save reusable SQL logic.

```
CREATE DATABASE IF NOT EXISTS company_db;
```

This ensures that company_db is created only if it doesn't already exist.

DROP

```
DROP DATABASE company_db;
```

This removes an entire database and all its tables permanently.

```
DROP DATABASE IF EXISTS company_db;
```

This prevents errors if the database does not exist.

```
DROP TABLE employees;
```

This removes a table and all its data.

```
DROP TABLE IF EXISTS employees;
```

This prevents an error if the table does not exist.

```
ALTER TABLE employees DROP COLUMN salary;
```

This removes the salary column from the employees table.

```
DROP INDEX idx_department ON employees;
```

This deletes the idx_department index on the employees table.

```
DROP VIEW high_salary_employees;
```

This removes a view.

```
DROP PROCEDURE GetEmployeesByDepartment;
```

This drops a stored procedure.

ALTER

```
ALTER TABLE employees  
ADD email VARCHAR(100);
```

This adds an email column to the employees table.

```
ALTER TABLE employees  
ADD phone_number VARCHAR(15),  
ADD address VARCHAR(255);
```

This adds multiple columns.

```
ALTER TABLE employees  
DROP COLUMN salary;
```

This permanently deletes the salary column.

```
ALTER TABLE employees  
DROP COLUMN IF EXISTS salary;
```

This permanently deletes the salary column and prevents errors if it doesn't exist.

```
ALTER TABLE employees  
MODIFY COLUMN salary DECIMAL(12,2);
```

This changes the salary column to store up to 12 digits with 2 decimal places.

```
ALTER TABLE employees  
ALTER COLUMN hire_date SET DEFAULT CURRENT_DATE;
```

This changes a column's default value.

```
ALTER TABLE employees  
CHANGE COLUMN old_column_name new_column_name VARCHAR(100);
```

Renames an existing column.

```
ALTER TABLE employees  
RENAME COLUMN old_column_name TO new_column_name;
```

Alternative (For PostgreSQL, MySQL 8+, SQL Server) to remove a column.

```
ALTER TABLE employees  
RENAME TO staff;
```

Renames a table.

```
ALTER TABLE employees  
ADD PRIMARY KEY (employee_id);
```

This ensures that employee_id is unique and cannot be NULL by adding a primary key.

```
ALTER TABLE employees
```

```
ADD CONSTRAINT fk_department
FOREIGN KEY (department_id) REFERENCES departments(department_id);
```

This links employees.department_id to departments.department_id by adding a foreign key.

```
ALTER TABLE employees
DROP CONSTRAINT fk_department;
```

This drops a foreign key.

```
ALTER TABLE employees
ADD INDEX idx_department (department);
```

This speeds up queries on the department column by adding an index.

```
ALTER TABLE employees
DROP INDEX idx_department;
```

This drops an index.

TRUNCATE

```
TRUNCATE TABLE employees;
```

This removes all rows from the employees table but keeps the table structure.

```
TRUNCATE TABLE employees;
```

When you truncate a table, the auto-increment counter is reset. In this example if employees has an AUTO_INCREMENT column (employee_id), the numbering will start again from 1.

```
SET FOREIGN_KEY_CHECKS = 0;
TRUNCATE TABLE employees;
SET FOREIGN_KEY_CHECKS = 1;
```

If the table has foreign key constraints, you may need to disable constraints before truncating. This allows truncation of tables with foreign key dependencies in MySQL.

```
START TRANSACTION;
```

```
TRUNCATE TABLE employees;
```

```
ROLLBACK; -- (This works in PostgreSQL, but not MySQL)
```

If supported, ROLLBACK will restore the deleted data. In MySQL, TRUNCATE cannot be rolled back. In PostgreSQL, TRUNCATE can be rolled back if inside a transaction.

PRIMARY KEY

A Primary Key (PK) is a column (or a set of columns) in a table that uniquely identifies each row. It ensures that the values in the key column(s) are unique and not NULL.

- Uniqueness - No two rows can have the same primary key value.
- Not NULL - A primary key column cannot have NULL values.
- One per Table - A table can have only one primary key.
- Automatic Indexing - Most databases automatically create an index on the primary key column for faster searches.

```
CREATE TABLE employees (  
  employee_id INT PRIMARY KEY,  
  first_name VARCHAR(50),  
  last_name VARCHAR(50)  
);
```

In this example `employee_id` is the primary key, ensuring uniqueness.

```
CREATE TABLE employees (  
  employee_id INT PRIMARY KEY AUTO_INCREMENT,  
  first_name VARCHAR(50),  
  last_name VARCHAR(50)  
);
```

In MySQL, you can use `AUTO_INCREMENT` to generate unique IDs automatically. In this example `AUTO_INCREMENT` ensures that each new row gets a unique `employee_id` (1, 2, 3, ...).

```
CREATE TABLE project_assignments (  
  employee_id INT,  
  project_id INT,  
  PRIMARY KEY (employee_id, project_id)
```

```
);
```

A composite primary key consists of two or more columns. This ensures that each combination of `employee_id` and `project_id` is unique.

```
ALTER TABLE employees  
ADD PRIMARY KEY (employee_id);
```

If a table already exists without a primary key, you can add one.

```
ALTER TABLE employees  
DROP PRIMARY KEY;
```

This drops a primary key. Dropping a primary key might affect foreign keys in other tables.

```
CREATE TABLE departments (  
    department_id INT PRIMARY KEY,  
    department_name VARCHAR(100)  
);  
  
CREATE TABLE employees (  
    employee_id INT PRIMARY KEY AUTO_INCREMENT,  
    first_name VARCHAR(50),  
    department_id INT,  
    FOREIGN KEY (department_id) REFERENCES departments(department_id)  
);
```

A foreign key in one table refers to the primary key of another table. In this example `employees.department_id` is a foreign key referring to `departments.department_id`.

FOREIGN KEY

A Foreign Key (FK) is a column (or a set of columns) in one table that refers to the Primary Key (PK) in another table. It establishes a relationship between the two tables and ensures referential integrity—meaning, the foreign key column can only contain values that exist in the referenced table.

- References a Primary Key - The foreign key column(s) must match values from a primary key in another table.
- Ensures Data Integrity - Prevents inserting invalid or orphaned records.
- Can Be NULL - If a foreign key is optional, it can be NULL.
- Can Have Multiple Foreign Keys - A table can have multiple foreign keys referencing different tables.

It's good to use indexes on foreign key columns for better performance. Be careful to avoid circular dependencies where tables reference each other.

```
CREATE TABLE departments (  
  department_id INT PRIMARY KEY,  
  department_name VARCHAR(100) NOT NULL  
);  
  
CREATE TABLE employees (  
  employee_id INT PRIMARY KEY AUTO_INCREMENT,  
  first_name VARCHAR(50) NOT NULL,  
  department_id INT,  
  FOREIGN KEY (department_id) REFERENCES departments(department_id)  
);
```

Here, `department_id` in the `employees` table is a foreign key that references `department_id` in `departments`. This ensures that every `department_id` in `employees` exists in `departments`.

```
ALTER TABLE employees  
ADD CONSTRAINT fk_department  
FOREIGN KEY (department_id) REFERENCES departments(department_id);
```

If a table already exists, you can add a foreign key like this.

```
ALTER TABLE employees  
ADD CONSTRAINT fk_department  
FOREIGN KEY (department_id) REFERENCES departments(department_id);
```

If a table already exists, you can add a foreign key like this.

```
ALTER TABLE employees
DROP FOREIGN KEY fk_department;
```

You can remove a foreign key constraint like this. In this example `fk_department` is the name of the foreign key constraint.

```
CREATE TABLE employees (
    employee_id INT PRIMARY KEY AUTO_INCREMENT,
    first_name VARCHAR(50) NOT NULL,
    department_id INT,
    FOREIGN KEY (department_id) REFERENCES departments(department_id)
    ON DELETE CASCADE
    ON UPDATE CASCADE
);
```

Foreign keys can define what happens when the referenced row in the parent table is deleted or updated.

- ON DELETE CASCADE - Deletes related rows in the child table when the parent row is deleted.
- ON DELETE SET NULL - Sets the foreign key column to NULL if the parent row is deleted.
- ON DELETE RESTRICT - Prevents deletion of a parent row if related rows exist.
- ON UPDATE CASCADE - Updates foreign key values in child rows if the primary key in the parent table changes.

```
CREATE TABLE customers (
    customer_id INT PRIMARY KEY,
    customer_name VARCHAR(100) NOT NULL
);
```

```
CREATE TABLE employees (
    employee_id INT PRIMARY KEY AUTO_INCREMENT,
    first_name VARCHAR(50) NOT NULL
);
```

```
CREATE TABLE orders (
    order_id INT PRIMARY KEY AUTO_INCREMENT,
    customer_id INT,
```



```
employee_id INT,  
FOREIGN KEY (customer_id) REFERENCES customers(customer_id) ON DELETE CASCADE,  
FOREIGN KEY (employee_id) REFERENCES employees(employee_id) ON DELETE SET NULL  
);
```

In this example orders table references both customers and employees. If a customer is deleted, their orders are deleted (CASCADE). If an employee is deleted, their orders remain but the employee_id is set to NULL.

UNIQUE

The UNIQUE constraint ensures that all values in a column (or combination of columns) are distinct—meaning no two rows can have the same value in that column. Unlike PRIMARY KEY, a table can have multiple UNIQUE constraints, and UNIQUE columns can contain NULL values (except in some databases like SQL Server, where only one NULL is allowed).

```
CREATE TABLE employees (  
    employee_id INT PRIMARY KEY AUTO_INCREMENT,  
    email VARCHAR(100) UNIQUE,  
    phone_number VARCHAR(15) UNIQUE  
);
```

This creates a UNIQUE constraint on a single column. In this example both email and phone_number must be unique. If you try inserting two employees with the same email or phone_number, the database will return an error.

```
CREATE TABLE employees (  
    employee_id INT PRIMARY KEY AUTO_INCREMENT,  
    first_name VARCHAR(50),  
    last_name VARCHAR(50),  
    department VARCHAR(50),  
    UNIQUE (first_name, last_name, department)  
);
```

A composite UNIQUE constraint ensures that the combination of values in multiple columns is unique. In this example the combination of (first_name, last_name, department) must be unique, but the individual columns can contain duplicate values.

```
ALTER TABLE employees  
ADD CONSTRAINT unique_email UNIQUE (email);
```

If a table already exists, you can add a UNIQUE constraint like this.

```
ALTER TABLE employees
DROP CONSTRAINT unique_email;
```

-- If you're using MySQL, use:

```
ALTER TABLE employees
DROP INDEX unique_email;
```

To drop a UNIQUE constraint, you need to know its constraint name.

```
INSERT IGNORE INTO employees (email, phone_number)
VALUES ('john@example.com', '1234567890');
```

If you try inserting duplicate values, you'll get an error. To avoid this, use INSERT IGNORE, REPLACE, or ON DUPLICATE KEY UPDATE. In this example using INSERT IGNORE (MySQL) make the database ignore duplicate entries instead of causing an error.

```
INSERT INTO employees (email, phone_number)
VALUES ('john@example.com', '1234567890')
ON DUPLICATE KEY UPDATE phone_number = '0987654321';
```

In this example using ON DUPLICATE KEY UPDATE (MySQL) makes the database update phone_number instead of failing if email already exists.

NOT NULL

The NOT NULL constraint ensures that a column cannot contain NULL values. It forces the column to always have a valid (non-null) value when inserting or updating data.

```
CREATE TABLE employees (
  employee_id INT PRIMARY KEY AUTO_INCREMENT,
  first_name VARCHAR(50) NOT NULL,
  last_name VARCHAR(50) NOT NULL,
  email VARCHAR(100) UNIQUE NOT NULL
);
```

When creating a table, you can specify NOT NULL to enforce that a column must have a value. In this example first_name, last_name, and email must always have values. If you try to insert a row without providing values for these columns, you'll get an error.

```
ALTER TABLE employees
MODIFY COLUMN first_name VARCHAR(50) NOT NULL;
```

If a column was created without NOT NULL, you can modify it. This ensures that first_name can never be NULL from now on. Important: Before applying NOT NULL, ensure the column does not contain NULL values, or the command will fail.

```
ALTER TABLE employees
MODIFY COLUMN first_name VARCHAR(50) NULL;
```

This way you can allow NULL values in a column. Now, first_name can store NULL values.

```
-- This works because all NOT NULL columns have values:
INSERT INTO employees (first_name, last_name, email)
VALUES ('John', 'Doe', 'john@example.com');

-- This will fail:
INSERT INTO employees (first_name, last_name)
VALUES ('John', 'Doe');
```

Error: Column 'email' cannot be null because email is defined as NOT NULL.

```
CREATE TABLE employees (
  employee_id INT PRIMARY KEY AUTO_INCREMENT,
  first_name VARCHAR(50) NOT NULL,
  salary DECIMAL(10,2) NOT NULL DEFAULT 50000.00
);
```

If a column is NOT NULL, you can set a default value to avoid NULL issues. In this example if no salary is provided, it defaults to 50,000.00.

CHECK

The CHECK constraint is used to enforce a rule on a column's values. It ensures that only valid data is inserted into the table by evaluating a specified condition. If the condition is not met, the database rejects the insertion or update. In MySQL versions before 8.0, CHECK constraints are ignored, so you should use triggers instead.

```
CREATE TABLE employees (
```

```
employee_id INT PRIMARY KEY AUTO_INCREMENT,  
first_name VARCHAR(50) NOT NULL,  
age INT CHECK (age >= 18),  
salary DECIMAL(10,2) CHECK (salary > 0)  
);
```

You can define a CHECK constraint when creating a table. In this example:

- age must be at least 18
- salary must be greater than 0
- if an attempt is made to insert an employee under 18 or with a salary of 0 or less, the database rejects it

```
ALTER TABLE employees  
ADD CONSTRAINT check_salary CHECK (salary > 0);
```

If the table is already created, you can add a CHECK constraint later.

```
ALTER TABLE employees  
DROP CONSTRAINT check_salary;
```

-- In MySQL, use:

```
ALTER TABLE employees  
DROP CHECK check_salary;
```

This drops a CHECK constraint.

```
CREATE TABLE employees (  
  employee_id INT PRIMARY KEY AUTO_INCREMENT,  
  first_name VARCHAR(50) NOT NULL,  
  age INT,  
  salary DECIMAL(10,2),  
  CHECK (age >= 18 AND salary > 30000)  
);
```

You can enforce multiple rules in a single CHECK constraint. In this example an employee must be at least 18 years old and have a salary greater than 30,000.

```
CREATE TABLE orders (  

```

```
order_id INT PRIMARY KEY AUTO_INCREMENT,  
quantity INT NOT NULL,  
price DECIMAL(10,2) NOT NULL,  
CHECK (quantity * price > 100)  
);
```

You can apply a CHECK constraint across multiple columns. This ensures that the total cost (quantity * price) is greater than 100.

DEFAULT

The DEFAULT constraint provides a predefined value for a column when no value is specified during insertion. It ensures that a column never remains empty unless explicitly set to NULL (if allowed).

```
CREATE TABLE employees (  
  employee_id INT PRIMARY KEY AUTO_INCREMENT,  
  first_name VARCHAR(50) NOT NULL,  
  salary DECIMAL(10,2) DEFAULT 50000.00,  
  hire_date DATE DEFAULT CURRENT_DATE  
);
```

When defining a table, you can specify a DEFAULT value for a column. In this example if no value is provided for salary, it defaults to 50,000.00 and hire_date defaults to the current date when a new employee is added.

```
-- This works because salary and hire_date have default values:  
INSERT INTO employees (first_name)  
VALUES ('John Doe');  
  
-- To explicitly override the default, provide a value:  
INSERT INTO employees (first_name, salary, hire_date)  
VALUES ('Jane Smith', 60000, '2025-01-01');
```

Inserting data with DEFAULT values

```
ALTER TABLE employees  
ALTER COLUMN salary SET DEFAULT 45000.00;
```

If a table already exists, you can add a DEFAULT value later. Now, if salary is not provided, it will default to 45,000.00.

```
ALTER TABLE employees
ALTER COLUMN salary DROP DEFAULT;
```

This removes a DEFAULT constraint.

```
-- Numeric default:
CREATE TABLE products (
  product_id INT PRIMARY KEY AUTO_INCREMENT,
  price DECIMAL(10,2) DEFAULT 9.99
);

-- Text default:
CREATE TABLE users (
  user_id INT PRIMARY KEY AUTO_INCREMENT,
  role VARCHAR(20) DEFAULT 'customer'
);

-- Boolean default (SQL Server, PostgreSQL):
CREATE TABLE accounts (
  account_id INT PRIMARY KEY AUTO_INCREMENT,
  is_active BOOLEAN DEFAULT TRUE
);

-- Date/Time default:
CREATE TABLE orders (
  order_id INT PRIMARY KEY AUTO_INCREMENT,
  order_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

DEFAULT with different data types

The INNER JOIN keyword is used to combine rows from two or more tables based on a matching condition. It returns only the rows where there is a match in both tables.

```
SELECT column_names
FROM table1
INNER JOIN table2
ON table1.common_column = table2.common_column;
```

In this example table1.common_column and table2.common_column represent the columns that must match between the two tables.

```
SELECT employees.name, departments.department_name, locations.location_name
FROM employees
INNER JOIN departments ON employees.department_id = departments.department_id
INNER JOIN locations ON departments.location_id = locations.location_id;
```

You can join more than two tables using INNER JOIN. This query retrieves the employee name, their department, and the location of the department.

LEFT JOIN

The LEFT JOIN (also called LEFT OUTER JOIN) retrieves all records from the left table and the matching records from the right table. If no match is found, it returns NULL for the columns from the right table.

```
SELECT column_names
FROM table1
LEFT JOIN table2
ON table1.common_column = table2.common_column;
```

In this example table1 is the left table (returns all its records) and table2 is the right table (returns only matching records, or NULL if no match is found).

```
SELECT employees.name, departments.department_name, locations.location_name
FROM employees
LEFT JOIN departments ON employees.department_id = departments.department_id
LEFT JOIN locations ON departments.location_id = locations.location_id;
```

You can join more than two tables using LEFT JOIN. This retrieves all employees, their department names, and locations if available.

RIGHT JOIN

The RIGHT JOIN (also called RIGHT OUTER JOIN) retrieves all records from the right table and the matching records from the left table. If no match is found, it returns NULL for the columns from the left table.

```
SELECT column_names
FROM table1
RIGHT JOIN table2
ON table1.common_column = table2.common_column;
```

In this example table1 is the left table (only matching records are included) and table2 is the right table (returns all its records, even if there's no match in table1).

```
SELECT employees.name, departments.department_name, locations.location_name
FROM employees
RIGHT JOIN departments ON employees.department_id = departments.department_id
RIGHT JOIN locations ON departments.location_id = locations.location_id;
```

You can join more than two tables using RIGHT JOIN. This retrieves all locations, all departments, and the employees if available.

FULL OUTER JOIN

The FULL OUTER JOIN (also called FULL JOIN) retrieves all records from both tables and matches them where possible. If there is no match, it returns NULL for the missing values.

```
SELECT column_names
FROM table1
FULL OUTER JOIN table2
ON table1.common_column = table2.common_column;
```

All rows from both tables are included. If a row has no match in the other table, NULL values appear in its columns.

```
SELECT employees.name, departments.department_name, locations.location_name
FROM employees
```



```
FULL OUTER JOIN departments ON employees.department_id = departments.department_id
FULL OUTER JOIN locations ON departments.location_id = locations.location_id;
```

This retrieves all employees, all departments, and all locations, filling in NULL where data is missing.

```
SELECT employees.employee_id, employees.name, departments.department_name
FROM employees
LEFT JOIN departments ON employees.department_id = departments.department_id

UNION

SELECT employees.employee_id, employees.name, departments.department_name
FROM employees
RIGHT JOIN departments ON employees.department_id = departments.department_id;
```

MySQL does not support FULL OUTER JOIN directly! You can simulate it using UNION of LEFT JOIN and RIGHT JOIN. The UNION ensures that all records from both tables are included.

CROSS JOIN

A CROSS JOIN creates a Cartesian product of two tables, meaning every row from the first table is combined with every row from the second table. No JOIN condition is required because it pairs every row from both tables. The total number of rows in the result set = (rows in table1) x (rows in table2).

CROSS JOIN can create large datasets!

- If both tables have 1000 rows each, the result will have 1,000,000 rows (1000 x 1000).
- Be cautious when using CROSS JOIN on large tables, as it can slow down queries.

```
SELECT column_names
FROM table1
CROSS JOIN table2;

-- OR (in some databases, you can use this alternative syntax):
SELECT column_names
FROM table1, table2;
```

This returns a Cartesian product where every row in table1 is combined with every row in table2.

```
SELECT employees.name, departments.department_name
FROM employees, departments;
```

If you accidentally forget to include an ON condition in a normal JOIN, it can behave like a CROSS JOIN! This creates a Cartesian product, which is usually not intended. Always use JOIN conditions (ON employees.department_id = departments.department_id) when necessary.

SELF JOIN

A SELF JOIN is a JOIN operation where a table is joined with itself. This is useful when comparing rows within the same table, such as hierarchical relationships (e.g., employees and managers) or finding duplicates.

```
SELECT A.column_name, B.column_name
FROM table_name A
JOIN table_name B
ON A.common_column = B.common_column;
```

The table is aliased as A and B to differentiate between the two instances. It works like a normal JOIN, but both instances refer to the same table.

UNION

The UNION operator is used to combine the results of two or more SELECT statements into a single result set. It removes duplicate rows by default.

```
SELECT column_names FROM table1
UNION
SELECT column_names FROM table2;
```

Both SELECT statements must have the same number of columns. The columns must have compatible data types. The result set removes duplicates unless you use UNION ALL.

```
SELECT employee_id, name, department FROM Employees_2023
UNION ALL
SELECT employee_id, name, department FROM Employees_2024;
```

UNION ALL keeps duplicates and is faster than UNION.

```
SELECT name, department FROM Employees_2023 WHERE department = 'IT'  
UNION  
SELECT name, department FROM Employees_2024 WHERE department = 'IT';
```

You can filter data in each SELECT statement. This will only include IT employees from both years.

```
SELECT employee_id, name FROM Employees_2023  
UNION  
SELECT employee_id, name FROM Employees_2024  
ORDER BY name ASC;
```

You can sort the final result using ORDER BY. The ORDER BY applies to the entire result set, not individual queries.

COUNT()

```
SELECT COUNT(*)  
FROM table_name;
```

This counts all rows in the table, including those with NULL values.

```
SELECT COUNT(column_name)  
FROM table_name;
```

This counts only non-null values in the specified column.

```
SELECT COUNT(DISTINCT department)  
FROM Employees;
```

If you want to count unique values in a column, use DISTINCT. NULL is ignored in DISTINCT.

```
SELECT department, COUNT(*) AS employee_count  
FROM Employees  
GROUP BY department;
```

You can use COUNT() with GROUP BY to count occurrences per category. NULL values are treated as a separate group.

```
SELECT COUNT(*)
```

```
FROM Employees
WHERE department = 'IT';
```

You can filter data before counting. This counts only employees in the IT department.

```
SELECT department, COUNT(*) AS employee_count
FROM Employees
GROUP BY department
HAVING COUNT(*) > 1;
```

If you need to filter after grouping, use HAVING. This only shows departments with more than one employee.

SUM()

```
SELECT SUM(column_name)
FROM table_name;
```

This adds up all numeric values in the specified column. NULL values are ignored in the summation.

```
SELECT customer, SUM(amount) AS total_spent
FROM Sales
GROUP BY customer;
```

You can use SUM() with GROUP BY to calculate totals per category.

```
SELECT SUM(amount)
FROM Sales WHERE customer = 'Alice';
```

You can filter the rows before calculating the sum.

```
SELECT customer, SUM(amount) AS total_spent
FROM Sales
GROUP BY customer
HAVING SUM(amount) > 100;
```

If you want to filter groups after aggregation, use HAVING.

```
SELECT SUM(DISTINCT amount)
```

```
FROM Sales;
```

If you want to sum only unique values, use DISTINCT. This adds only unique values in the amount column.

```
SELECT SUM(COALESCE(amount, 0))  
FROM Sales;
```

This treats NULL as 0 instead of ignoring it.

AVG()

```
SELECT AVG(column_name)  
FROM table_name;
```

This returns the average value of the column. NULL values are ignored in the calculation.

```
SELECT customer, AVG(amount) AS avg_spent  
FROM Sales  
GROUP BY customer;
```

You can use AVG() with GROUP BY to calculate averages per category. In this example each customer's average spending is calculated. If a customer has only NULL values, the result remains NULL.

```
SELECT AVG(amount)  
FROM Sales  
WHERE customer = 'Alice';
```

You can filter rows before calculating the average.

```
SELECT customer, AVG(amount) AS avg_spent  
FROM Sales  
GROUP BY customer  
HAVING AVG(amount) > 100;
```

If you want to filter groups after aggregation, use HAVING. In this example only customers who spent more than 100 on average are included.

```
SELECT AVG(DISTINCT amount)  
FROM Sales;
```

If you want to average only unique values, use DISTINCT. This calculates the average of unique values in the amount column.

```
SELECT AVG(COALESCE(amount, 0))  
FROM Sales;
```

This treats NULL as 0 instead of ignoring it.

MIN()

```
SELECT MIN(column_name)  
FROM table_name;
```

This returns the smallest value in the specified column. NULL values are ignored.

```
SELECT category, MIN(price) AS cheapest_item  
FROM Products  
GROUP BY category;
```

You can use MIN() with GROUP BY to find the smallest value per category. This finds the cheapest product in each category.

```
SELECT MIN(price)  
FROM Products  
WHERE category = 'Electronics';
```

You can filter rows before calculating the minimum. This finds the cheapest Electronics product.

```
SELECT category, MIN(price) AS cheapest_item  
FROM Products  
GROUP BY category  
HAVING MIN(price) > 500;
```

If you want to filter groups after aggregation, use HAVING. This only includes categories where the cheapest item is above \$500.

```
SELECT MIN(order_date) AS first_order  
FROM Orders;
```

The MIN() function also works with dates to find the earliest date. This finds the earliest order date in the table.

```
SELECT MIN(COALESCE(price, 0))  
FROM Products;
```

This treats NULL as 0 instead of ignoring it.

MAX()

```
SELECT MAX(column_name)  
FROM table_name;
```

This returns the largest value in the specified column. NULL values are ignored.

```
SELECT category, MAX(price) AS most_expensive  
FROM Products  
GROUP BY category;
```

You can use MAX() with GROUP BY to find the highest value per category. This finds the most expensive product in each category.

```
SELECT MAX(price)  
FROM Products  
WHERE category = 'Electronics';
```

You can filter rows before calculating the maximum. This finds the most expensive Electronics product.

```
SELECT category, MAX(price) AS most_expensive  
FROM Products  
GROUP BY category  
HAVING MAX(price) > 800;
```

If you want to filter groups after aggregation, use HAVING. This only includes categories where the most expensive item is above \$800.

```
SELECT MAX(order_date) AS latest_order  
FROM Orders;
```

The MAX() function also works with dates to find the latest date. This finds the latest order date in the table.

```
SELECT MAX(COALESCE(price, 0))
```

```
FROM Products;
```

This treats NULL as 0 instead of ignoring it.

GROUP BY

The GROUP BY clause is used to group rows that have the same values in specified columns into summary rows. It is commonly used with aggregate functions (SUM(), COUNT(), AVG(), MIN(), MAX()) to perform calculations for each group.

```
SELECT column_name, AGGREGATE_FUNCTION(column_name)
FROM table_name
GROUP BY column_name;
```

The GROUP BY clause follows the WHERE clause and comes before the ORDER BY clause.

```
SELECT customer, SUM(amount) AS total_spent
FROM Sales
GROUP BY customer;
```

Using GROUP BY to find total sales per customer.

```
SELECT customer, category, SUM(amount) AS total_spent
FROM Sales
GROUP BY customer, category;
```

You can group by multiple columns. The result is grouped by customer and category.

```
SELECT customer, COUNT(order_id) AS total_orders
FROM Sales
GROUP BY customer;
```

If you want to count the number of orders per customer. COUNT(order_id) counts how many orders each customer made.

```
SELECT customer, SUM(amount) AS total_spent
FROM Sales
GROUP BY customer
HAVING SUM(amount) > 250;
```


The HAVING clause is used to filter groups after aggregation. Only customers who spent more than 250 are included. WHERE cannot be used with aggregate functions, so we use HAVING.

```
SELECT customer, SUM(amount) AS total_spent
FROM Sales
GROUP BY customer
ORDER BY total_spent DESC;
```

This groups by customer and sorts results in descending order of total spent.

HAVING

The HAVING clause is used to filter grouped records after applying aggregate functions like SUM(), COUNT(), AVG(), MIN(), or MAX(). Unlike WHERE, which filters individual rows before aggregation, HAVING filters after aggregation.

```
SELECT column_name, AGGREGATE_FUNCTION(column_name)
FROM table_name
GROUP BY column_name
HAVING condition;
```

HAVING is always used with GROUP BY. You cannot use HAVING without an aggregate function.

```
SELECT customer, SUM(amount) AS total_spent
FROM Sales
GROUP BY customer
HAVING SUM(amount) > 200;
```

This example we search for customers who spent more than \$200.

```
-- Using WHERE (filters before grouping)
SELECT customer, SUM(amount) AS total_spent
FROM Sales
WHERE amount > 100
GROUP BY customer;
-- This filters out rows where amount <= 100 before grouping
```

```
-- Using HAVING (filters after aggregation)
SELECT customer, SUM(amount) AS total_spent
FROM Sales
GROUP BY customer
HAVING SUM(amount) > 200;
-- This calculates total sales first, then filters customers with total sales > 200.
```

```
SELECT category, SUM(amount) AS total_sales
FROM Sales
GROUP BY category
HAVING SUM(amount) > 100 AND COUNT(order_id) > 1;
```

You can use multiple conditions with HAVING using AND/OR. In this example only categories where total sales > 100 AND at least 2 orders exist are included.

```
SELECT customer, SUM(amount) AS total_spent
FROM Sales
GROUP BY customer
HAVING SUM(amount) > 100
ORDER BY total_spent DESC;
```

You can combine HAVING with ORDER BY to sort the filtered results. This filters customers who spent more than 100 and sorts them in descending order.

COALESCE

The COALESCE() function returns the first non-null value from a list of expressions. It is commonly used to handle NULL values by providing a default value when NULL is encountered.

```
SELECT COALESCE(expression1, expression2, ..., default_value);
```

In this example we check each value from left to right and return the first non-null value. If all values are NULL, it returns the default_value.

```
SELECT name, COALESCE(phone, 'No Phone Available') AS contact_info
FROM Customers;
```

In this example we replace NULL with 'No Phone Available'.

```
SELECT COALESCE(NULL, NULL, 'First Non-Null', 'Second Non-Null') AS result;
```

You can use multiple values in COALESCE(). SQL checks values from left to right and returns the first non-null value.

```
SELECT name, amount, COALESCE(amount, 0) * 1.1 AS updated_amount  
FROM Payments;
```

If a column has NULL values, mathematical operations may fail. COALESCE() helps by providing a default numeric value. This replaces NULL in amount with 0 before performing multiplication.

```
SELECT department, COALESCE(SUM(salary), 0) AS total_salary  
FROM Employees  
GROUP BY department;
```

When using aggregate functions, NULL values may cause issues. COALESCE() ensures default values. In this example if a department has no employees, SUM(salary) would be NULL, but COALESCE makes it 0.

```
-- Returns 'Hello'  
SELECT COALESCE(NULL, NULL, 'Hello', 'World');  
  
-- Returns 'Fallback'  
SELECT IFNULL(NULL, 'Fallback');
```

Difference between COALESCE() and IFNULL().

Subqueries and nested queries

Subqueries, also known as nested queries, are queries inside another query, enclosed in parentheses () and used within SELECT, FROM, or WHERE clauses.. They are useful when you need to fetch data that depends on the results of another query.

Types of Subqueries:

- Single-row subquery - Returns one value.
- Multi-row subquery - Returns multiple values.
- Correlated subquery - Depends on the outer query and runs once for each row in the outer query.
- Nested subquery - A subquery inside another subquery.

```
SELECT employee_name, salary,
```

```
(SELECT MAX(salary) FROM Employees) AS highest_salary  
FROM Employees;
```

This example returns the highest salary in the company.

```
SELECT employee_name, salary  
FROM Employees  
WHERE salary > (SELECT AVG(salary) FROM Employees);
```

This example finds employees earning above the company's average salary.

```
SELECT *  
FROM (SELECT employee_name, salary FROM Employees WHERE salary > 50000) AS HighEarners;
```

This example creates a temporary table to store employees with salaries above \$50,000.

```
SELECT employee_name, department_id, salary  
FROM Employees e1  
WHERE salary = (  
    SELECT MAX(salary)  
    FROM Employees e2  
    WHERE e1.department_id = e2.department_id  
);
```

A correlated subquery executes once per row of the outer query. This example finds employees with the highest salary in their department. The subquery runs for each row in the outer query. It finds the maximum salary within the same department.

```
SELECT employee_name, salary, department_id  
FROM Employees  
WHERE salary = (  
    SELECT MIN(salary)  
    FROM Employees  
    WHERE department_id = (  
        SELECT department_id  
        FROM Employees  
        GROUP BY department_id  
        ORDER BY AVG(salary) DESC
```

```
        LIMIT 1
    )
);
```

This nested subquery finds the employee with the lowest salary in the department with the highest average salary. The innermost query finds the department with the highest average salary. The middle query finds the lowest salary in that department. The outer query selects the employee with that salary.

Advantages of subqueries:

- Improves readability by breaking down complex queries.
- Reduces joins when filtering data.
- Can be used in SELECT, FROM, and WHERE clauses.

Use JOINS instead of subqueries when possible because:

- Joins are faster in many cases.
- Subqueries can be inefficient, especially correlated subqueries.

Indexes

An index in SQL is a database object that speeds up query performance by allowing faster lookups of rows. It works like an index in a book helping you find information quickly without scanning every page. Instead of scanning every row in a table (full table scan), the database uses the index to locate data faster.

```
CREATE TABLE Employees (
  employee_id INT PRIMARY KEY, -- This automatically creates an index
  name VARCHAR(100),
  salary DECIMAL(10,2)
);
```

A primary index created automatically when a PRIMARY KEY is defined and ensures uniqueness and fast lookups for primary key searches.

```
CREATE UNIQUE INDEX idx_employee_email
ON Employees(email);
```

A unique index ensures all values in the indexed column are unique. It's useful for columns like email, username, or phone number.

```
CREATE INDEX idx_employee_name_salary
```

```
ON Employees(name, salary);
```

```
-- Efficient for queries like:
```

```
SELECT * FROM Employees WHERE name = 'Alice' AND salary > 50000;
```

A composite index indexes two or more columns together. It improves performance when filtering by both columns.

```
CREATE FULLTEXT INDEX idx_article_content
```

```
ON Articles(content);
```

A full-text index is used for fast searching in large text fields like articles, blogs, descriptions. It works best for search queries (LIKE '%keyword%' is slow).

```
CREATE CLUSTERED INDEX idx_employee_salary
```

```
ON Employees(salary);
```

A clustered index rearranges physical data order. Each table can have only ONE clustered index. It stores rows in physical order based on the indexed column. By default, PRIMARY KEY is a clustered index.

```
CREATE NONCLUSTERED INDEX idx_employee_name
```

```
ON Employees(name);
```

```
-- Efficient for queries like:
```

```
SELECT * FROM Employees WHERE name = 'Alice';
```

A non-clustered index points to data instead of rearranging it. It's faster than clustered indexes when searching specific values. It stores a separate index structure that points to actual table rows.

```
-- MySQL:
```

```
SHOW INDEX
```

```
FROM Employees;
```

```
-- In PostgreSQL:
```

```
SELECT *
```

```
FROM pg_indexes
```

```
WHERE tablename = 'employees';
```

Viewing existing indexes.

```
-- In PostgreSQL:  
DROP INDEX idx_employee_name;
```

```
-- In MySQL:  
ALTER TABLE Employees  
DROP INDEX idx_employee_name;
```

Removing an index.

Stored procedures and functions

Stored procedures and functions are reusable blocks of SQL code that help automate and optimize database operations. They reduce redundancy, improve performance, and enhance security by minimizing direct access to tables.

- Reusable - Write once, execute multiple times.
- Can accept parameters - Dynamic operations.
- Can execute multiple SQL statements - SELECT, INSERT, UPDATE, DELETE.
- Faster execution - Precompiled and optimized.
- Enhances security - Restricts direct table access.

```
-- Creating a stored procedure:  
DELIMITER $$ -- Used in MySQL to change the delimiter  
CREATE PROCEDURE GetEmployeeDetails(IN emp_id INT)  
BEGIN  
    SELECT * FROM Employees WHERE employee_id = emp_id;  
END $$  
DELIMITER ; -- Reset delimiter
```

```
-- Executing the stored procedure:  
CALL GetEmployeeDetails(101);
```

In this example we create and then call a procedure to get employee details.

```
-- Creating a stored procedure:  
DELIMITER $$
```

```

CREATE PROCEDURE IncreaseSalary(IN emp_id INT, IN increment DECIMAL(10,2))
BEGIN
    UPDATE Employees
    SET salary = salary + increment
    WHERE employee_id = emp_id;
END $$
DELIMITER ;

-- Executing the stored procedure:
CALL IncreaseSalary(101, 5000);

```

In this example we create and call a stored procedure with multiple statements that increases salary based on employee ID.

A function in SQL is similar to a stored procedure, but it must return a value and can be used inside SELECT statements.

- Must return a single value.
- Cannot modify data (no INSERT, UPDATE, DELETE).
- Can be used in SELECT statements.

```

-- Creating a function:
DELIMITER $$
CREATE FUNCTION GetEmployeeSalary(emp_id INT) RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    DECLARE salary DECIMAL(10,2);
    SELECT salary INTO salary FROM Employees WHERE employee_id = emp_id;
    RETURN salary;
END $$
DELIMITER ;

-- Using the function in a query:
SELECT GetEmployeeSalary(101) AS EmployeeSalary;

```

In this example we create a function to get employee salary.

```

-- Dropping a stored procedure

```



```
DROP PROCEDURE IF EXISTS GetEmployeeDetails;
```

```
-- Dropping a function
```

```
DROP FUNCTION IF EXISTS GetEmployeeSalary;
```

This is how you can drop a stored procedure or function.

Use a stored procedure when you need to:

- Perform multiple operations (e.g., Insert + Update).
- Execute a task that doesn't return a value.
- Improve security by restricting direct table access.

Use a function when you need to:

- Compute and return a single value.
- Use it inside a SELECT statement.

Views

A view is a virtual table that stores a saved SQL query. It does not store actual data but provides a way to simplify complex queries and secure data access by restricting direct table access. Advantages:

- Simplifies Complex Queries - Save frequently used queries as a view.
- Enhances Security - Restrict access to specific columns/rows.
- Provides Abstraction - Users see a virtual table without accessing raw data.
- Improves Maintainability - Change the view definition instead of rewriting queries.

Disadvantages:

- Performance Overhead - Each time a view is accessed, the query runs.
- Cannot Modify Some Views - Views with multiple tables or aggregations cannot be updated.
- Storage Usage (Materialized Views) - Some databases store physical copies (not standard views).

```
-- Creating a view:
```

```
CREATE VIEW EmployeeView AS
```

```
SELECT employee_id, name, department, salary
```

```
FROM Employees
```

```
WHERE salary > 50000;
```

```
-- Retrieving data:
```

```
SELECT * FROM EmployeeView;
```

Creating and calling a view for employee details. In this example the database executes the query whenever you access EmployeeView. You don't need to rewrite the SELECT statement every time.

```
CREATE OR REPLACE VIEW EmployeeView AS  
SELECT employee_id, name, department, salary  
FROM Employees  
WHERE department = 'IT';
```

Use CREATE OR REPLACE to modify a view. In this example we update a view to show only the IT department.

```
DROP VIEW EmployeeView;
```

This is how you can delete a view.

```
CREATE VIEW EmployeeDepartmentView AS  
SELECT e.employee_id, e.name, d.department_name, e.salary  
FROM Employees e  
JOIN Departments d ON e.department_id = d.department_id;
```

In this example we create a complex view with a join.

```
UPDATE EmployeeView  
SET salary = 60000  
WHERE employee_id = 101;
```

You CAN update data through a view if:

- The view is based on a single table.
- The view does not use aggregations (SUM, COUNT, AVG, etc.).
- The view does not use DISTINCT, GROUP BY, or JOINS.

You CANNOT update a view if:

- It contains JOINS, GROUP BY, or aggregate functions.
- It has DISTINCT, UNION, or calculated columns.

Transactions and concurrency control

A transaction is a sequence of one or more SQL statements executed as a single unit of work. Transactions ensure data consistency by following the ACID properties:

- Atomicity - All or nothing; either the transaction is fully completed, or none of it happens.
- Consistency - The database remains in a valid state before and after the transaction.
- Isolation - Multiple transactions can run at the same time without interfering with each other.
- Durability - Once a transaction is committed, the changes are permanent even if the system crashes.

```
START TRANSACTION;
```

Beginning a transaction.

```
COMMIT;
```

Saving all changes made in the transaction.

```
ROLLBACK;
```

Reverting all changes if something goes wrong

```
SAVEPOINT save1;
```

Creating a savepoint.

```
ROLLBACK TO save1;
```

Rolling back only to the savepoint.

```
-- Without a transaction (risky):  
UPDATE Accounts SET balance = balance - 500 WHERE name = 'Alice';  
UPDATE Accounts SET balance = balance + 500 WHERE name = 'Bob';  
  
-- Using a transaction (safe):  
START TRANSACTION;  
  
UPDATE Accounts SET balance = balance - 500 WHERE name = 'Alice';  
UPDATE Accounts SET balance = balance + 500 WHERE name = 'Bob';
```

```
COMMIT; -- Ensures both updates are saved together
```

In this example we transfer \$500 from Alice to Bob. Without a transaction if the system crashes between the two updates, money might be deducted from Alice but not added to Bob. But using a transaction the two queries are done together.

Concurrency control ensures that multiple transactions execute correctly when they happen simultaneously (e.g., multiple users making bank transfers). Problems in concurrency control:

- Dirty Read - One transaction reads uncommitted changes from another.
- Lost Update - Two transactions update the same data, causing one change to be lost.
- Non-Repeatable Read - A value changes between two reads in the same transaction.
- Phantom Read - A new row appears/disappears when a transaction is re-run.

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

Setting an isolation level.

Isolation levels prevent concurrency issues by controlling how transactions interact.

Isolation level	Prevents dirty reads?	Prevents non-repeatable reads?	Prevents phantom reads?
Read uncommitted	No	No	No
Read committed	Yes	No	No
Repeatable read	Yes	Yes	No
Serializable	Yes	Yes	Yes

```
LOCK TABLE Accounts IN EXCLUSIVE MODE;
```

Databases use locks to prevent concurrent issues:

- Shared lock (read lock) - Allows multiple reads, but prevents writes.
- Exclusive lock (write lock) - Prevents other transactions from reading or writing.

Normalization and database design

Normalization is the process of organizing data in a database to reduce redundancy and improve data integrity. It ensures that data is stored efficiently while maintaining consistency. The goal of normalization is to eliminate data duplication and ensure logical data dependencies.

Normalization is important because it:

- Eliminates data redundancy - Saves storage space.
- Ensures data consistency - Avoids data anomalies.
- Improves data integrity - Enforces relationships properly.
- Easier to maintain and update - Fewer errors in the database.

Normalization is done through a series of stages called normal forms (NF). Each higher normal form builds on the previous one, adding more refinement.

Best Practices for database design:

- Use Primary & Foreign Keys - Maintain relationships properly.
- Normalize to at least 3NF - Avoid redundancy & anomalies.
- Use Indexing - Speed up query performance.
- Plan for Scalability - Design for future data growth.
- Consider Denormalization for Performance - Only when needed.

First Normal Form (1NF)

First Normal Form (1NF) is the foundation of database normalization. It ensures that a table is structured properly by eliminating duplicate and multi-valued attributes.

A table is in 1NF if it follows these rules:

- **Each column contains only single, indivisible values (no lists or arrays).**
- **Each column should have a unique identifier (Primary Key).**
- **No multiple values in a single column.**
- **Row order is not used to convey information.**

Example, Unnormalized Table (UNF - UnNormalized Form):

StudentID	StudentName	Courses	Instructor	Fee
101	Alice	Math, English	Smith, Johnson	100, 80
102	Bob	Math	Smith	100

This table has repeating groups in the Courses, Instructor, and Fee columns.

Table in 1NF:

StudentID	StudentName	Course	Instructor	Fee
101	Alice	Math	Smith	100

StudentID	StudentName	Course	Instructor	Fee
101	Alice	English	Johnson	80
102	Bob	Math	Smith	100

We eliminated repeating groups, now each field contains only a single value (atomic values). Also we have a unique key to indentify rows as StudentID and Course columns are set as a composite key.

But we still have redundancy (e.g., Alice's name is repeated), and the Fee depends on Course, not Student-Course combo.

Second Normal Form (2NF)

Second Normal Form (2NF) is a step further in database normalization that ensures no partial dependencies exist in a table. (A partial dependency is when a column depends on only a part of a composite primary key instead of the whole key.) It eliminates redundant data by splitting tables and establishing relationships.

A table is in 2NF if:

- It is already in 1NF.
- **All non-key columns depend on the entire primary key (not just part of it).**

In our previous 1NF example:

- StudentName depends only on StudentID
- Fee and Instructor depend only on Course

So to make it 2NF we split it into 3 tables:

Students table:

StudentID	StudentName
101	Alice
102	Bob

Courses table:

Course	Instructor	Fee
Math	Smith	100
English	Johnson	80

Enrollment table:

StudentID	Course
101	Math
101	English
102	Math

Now all non-key attributes depend on the whole key, and redundancy is reduced.

Third Normal Form (3NF)

Third Normal Form (3NF) is the next level of database normalization that ensures no transitive dependencies exist in a table. (A transitive dependency is if column A depends on column B, and column B depends on column C, then A depends transitively on C.) It helps to further reduce redundancy and improve data integrity.

A table is in 3NF if:

- It is already in 2NF.
- **A non-key column should not depend on another non-key column.**

Look closely at the Courses table in the 2NF example.

Now ask yourself:

Is everything in this table only dependent on the primary key Course?

If yes, then our design is in 3NF.

But let's say you learn something new:

"Each instructor is assigned a standard fee rate so the course's fee should be based on the instructor."

This introduces a transitive dependency: Course → Instructor → Fee

This violates 3NF as a non-key column (Fee) depends on another non-key column (Instructor), so we fix it by separating the Instructor:

Courses table:

Course	Instructor
Math	Smith
English	Johnson

Instructors table:

Instructor	Fee
Smith	100
Johnson	80

Now:

- Course → Instructor
- Instructor → Fee
- No transitive dependencies

Boyce-Codd Normal Form (BCNF)

BCNF is an advanced version of 3NF, ensuring even stricter rules. If there are multiple candidate keys, each must uniquely determine all attributes. Most databases follow 3NF or BCNF to balance efficiency & performance.

Denormalization

While normalization reduces redundancy, it can sometimes slow down queries due to joins.

- Denormalization is the process of reintroducing redundancy to improve performance.
- This is useful for read-heavy applications like data warehouses.

When to Use denormalization?

- Faster query performance (avoiding multiple joins).
- Reporting & analytics (precomputed summaries).
- Cache data for quick retrieval.

SQL performance optimization

Indexes speed up data retrieval by creating a sorted structure for searching. However, they slow down INSERT, UPDATE, and DELETE operations because they need to be updated.

How to use indexing effectively?

- Use indexes on columns that are frequently searched or filtered (e.g., WHERE, JOIN, ORDER BY).
- Avoid indexing small tables (full scans are often faster).
- Use composite indexes for queries filtering multiple columns.

```
SELECT id, name FROM Employees; -- Faster
SELECT * FROM Employees; -- Slower
```

Avoid SELECT * (Fetch only required columns)

```
SELECT department, COUNT(*)
FROM Employees
WHERE department = 'IT'
GROUP BY department;
```

Use WHERE instead of HAVING (when possible). HAVING filters after grouping, making it slower than WHERE.

```
SELECT * FROM Orders LIMIT 1000;
```

Use LIMIT for large datasets. This prevents the query from retrieving unnecessary data.

```
CREATE INDEX idx_employee_department_id ON Employees(department_id);
CREATE INDEX idx_department_id ON Departments(department_id);

SELECT e.name, d.department_name
FROM Employees e
JOIN Departments d ON e.department_id = d.department_id;
```

Optimize JOIN operations. In this example if department_id is not indexed, this join will be slow.

Choose the Right JOIN Type:

- INNER JOIN - Faster than OUTER JOINS (fetches only matching rows).
- LEFT JOIN - If needed, filter unnecessary NULLs using WHERE column IS NOT NULL.

```
-- Bad (using IN with subquery - slower):
SELECT * FROM Employees
WHERE department_id IN (SELECT department_id FROM Departments WHERE active = 1);
```

```
-- Good (using EXISTS - faster):
```

```
SELECT * FROM Employees e
WHERE EXISTS (SELECT 1 FROM Departments d WHERE e.department_id = d.department_id AND d.active = 1);
```

Use EXISTS instead of IN because EXISTS stops scanning once it finds a match, whereas IN processes the entire subquery.

```
-- Example of bad data types:
```

```
CREATE TABLE Orders (
  order_id VARCHAR(255), -- Bad, should be INT
  amount FLOAT -- Bad, should be DECIMAL(10,2)
);
```

```
-- Optimized data types:
```

```
CREATE TABLE Orders (
  order_id INT PRIMARY KEY, -- Faster
  amount DECIMAL(10,2) -- Accurate
);
```

Use proper data types. Choosing the right data type helps in reducing storage size and query execution time.

```
CREATE TABLE OrderSummary (
  order_id INT PRIMARY KEY,
  customer_name VARCHAR(100),
  total_price DECIMAL(10,2) -- Pre-calculated total instead of joining multiple tables
);
```

Use proper normalization & denormalization. In this example instead of doing multiple JOINS for an e-commerce order, we store frequently accessed data in one table.

Use query caching to avoid recomputing:

- Enable query cache - If the same queries run frequently.
- Use materialized views - Store results of complex calculations for re-use.
- Cache results in application layer - Instead of hitting the database every time.

```
CREATE TABLE Orders (
  order_id INT,
  order_date DATE
```

```
) PARTITION BY RANGE (YEAR(order_date)) (  
    PARTITION p2019 VALUES LESS THAN (2020),  
    PARTITION p2020 VALUES LESS THAN (2021),  
    PARTITION p2021 VALUES LESS THAN (2022)  
);
```

Partition large tables for faster queries. If a table grows too large, partitioning helps in improving performance. In this example we are range partitioning our Orders table, which allows queries on 2020 data to skip scanning older partitions.

```
-- Bad (nested subquery - slower):  
SELECT * FROM Employees  
WHERE department_id = (SELECT department_id FROM Departments WHERE department_name = 'IT');  
  
-- Good (using JOIN - faster):  
SELECT e.* FROM Employees e  
JOIN Departments d ON e.department_id = d.department_id  
WHERE d.department_name = 'IT';
```

Avoid too many subqueries. JOINS are usually optimized better by the SQL engine.

```
EXPLAIN ANALYZE  
SELECT *  
FROM Employees  
WHERE name = 'John Doe';
```

Monitor query performance. Use EXPLAIN (MySQL, PostgreSQL) or EXPLAIN ANALYZE (PostgreSQL) to check how a query is executed and optimized.

It shows:

- Indexes used or missing
- Query execution time
- Full table scans (bad!)

```
INSERT INTO Orders (order_id, amount)  
VALUES (101, 200), (102, 300), (103, 150);
```

Use batch inserts instead of single inserts. Single insert queries are slower than batched inserts.

```
START TRANSACTION;  
UPDATE Orders  
SET amount = amount * 1.1  
WHERE customer_id = 5;  
COMMIT;
```

Use transactions for bulk updates. It reduces disk writes compared to individual updates.

```
OPTIMIZE TABLE Employees;
```

Rebuild indexes periodically, especially for frequently updated tables.

```
ANALYZE TABLE Orders;
```

Use VACUUM (PostgreSQL) or ANALYZE (MySQL) to optimize performance.

Common Table Expressions (CTE) and recursive queries

A Common Table Expression (CTE) is a temporary result set that exists only during the execution of a query. It makes complex queries more readable and helps in recursive queries.

- Improves Readability - Breaks complex queries into simpler parts.
- Reusable - You can reference a CTE multiple times in the same query.
- Better Performance - Helps with optimization by avoiding repeated calculations.
- Supports Recursion - Essential for hierarchical data (e.g., organizational trees).

```
WITH cte_name AS (  
    -- CTE Query  
    SELECT column1, column2 FROM SomeTable WHERE condition  
)  
SELECT * FROM cte_name;
```

Basic syntax of a CTE.

```
-- Without CTE (messy query):  
SELECT e.name, d.department_name, COUNT(*) AS total_employees  
FROM Employees e
```

```

JOIN Departments d ON e.department_id = d.department_id
WHERE d.active = 1
GROUP BY e.name, d.department_name
HAVING COUNT(*) > 5;

-- Using a CTE (better structure):
WITH ActiveDepartments AS (
    SELECT department_id, department_name FROM Departments WHERE active = 1
)
SELECT e.name, d.department_name, COUNT(*) AS total_employees
FROM Employees e
JOIN ActiveDepartments d ON e.department_id = d.department_id
GROUP BY e.name, d.department_name
HAVING COUNT(*) > 5;

```

The CTE ActiveDepartments filters active departments first. The main query is cleaner and focuses on employees.

```

WITH RecursiveCTE AS (
    -- Base Case (Starting Point)
    SELECT column1, column2 FROM Table WHERE condition

    UNION ALL

    -- Recursive Case (Refers to Itself)
    SELECT column1, column2 FROM Table
    JOIN RecursiveCTE ON Table.some_id = RecursiveCTE.some_id
)
SELECT * FROM RecursiveCTE;

```

A recursive CTE is used when dealing with hierarchical data, like:

- Employee-manager relationships (Org Charts)
- Category-subcategory structures
- Family trees
- Graph traversal

```
WITH RECURSIVE EmployeeHierarchy AS (  
  SELECT employee_id, name, manager_id, 1 AS level  
  FROM Employees  
  WHERE employee_id = 1  
  
  UNION ALL  
  
  SELECT e.employee_id, e.name, e.manager_id, eh.level + 1  
  FROM Employees e  
  JOIN EmployeeHierarchy eh ON e.manager_id = eh.employee_id  
  WHERE eh.level < 5 -- Stop at 5 levels deep  
)  
SELECT * FROM EmployeeHierarchy;
```

If a table has circular references (e.g., Employee reports to themselves), the recursion may run indefinitely. In this example we prevent this by using a WHERE clause.

Window functions

Window functions perform calculations across a specific subset of rows, called a window (or partition), while still keeping each row in the result set. Unlike GROUP BY, which collapses rows into a single result per group, window functions retain individual rows while applying aggregations, rankings, or running totals.

```
FUNCTION() OVER (  
  -- Divides the dataset into groups (optional):  
  PARTITION BY column_name  
  -- Defines the order for calculations (optional):  
  ORDER BY column_name  
)
```

Basic syntax of a window function.

Category	Function	Use Case
Ranking	ROW_NUMBER()	Assigns unique row numbers

Category	Function	Use Case
	RANK()	Ranks with gaps for ties
	DENSE_RANK()	Ranks without gaps
Aggregation	SUM()	Running totals
	AVG()	Moving averages
	COUNT()	Counts rows per partition
Offset	LAG()	Previous row's value
	LEAD()	Next row's value
Cumulative	FIRST_VALUE()	First row's value in partition
	LAST_VALUE()	Last row's value in partition
	NTILE(n)	Divides data into n buckets

Types of window functions.

```
SELECT name, department, salary,
       ROW_NUMBER() OVER (PARTITION BY department ORDER BY salary DESC) AS row_num
FROM Employees;
```

In this example each row gets a unique number, regardless of ties:

name	department	salary	row_num
Alice	IT	7000	1
Bob	IT	6000	2
Carol	IT	6000	3
Dave	HR	5000	1
Eve	HR	4000	2

```
SELECT name, department, salary,
       RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS rank_num
FROM Employees;
```

Ranks with gaps when values tie:

name	department	salary	rank_num
Alice	IT	7000	1
Bob	IT	6000	2
Carol	IT	6000	2

name	department	salary	rank_num
Dave	HR	5000	1
Eve	HR	4000	2

```
SELECT
  Region,
  Salesperson,
  SaleAmount,
  SUM(SaleAmount) OVER (
    PARTITION BY Region
  ) AS TotalPerRegion
FROM Sales;
```

This example sums all rows within each partition (region) and repeats the total, returns:

Region	Salesperson	SaleAmount	TotalPerRegion
East	Alice	100	450
East	Bob	150	450
East	Carol	200	450
West	Dave	300	550
West	Eve	250	550

JSON and XML handling

Many databases (MySQL, PostgreSQL, SQL Server) have native JSON support. JSON data can be stored in text columns (VARCHAR, TEXT) or JSON type columns. You can query and manipulate JSON fields directly.

```
CREATE TABLE Employees (
  id INT PRIMARY KEY,
  name VARCHAR(50),
  details JSON -- Stores structured JSON data
);

INSERT INTO Employees (id, name, details)
```



```
VALUES (1, 'Alice', '{"age": 30, "position": "Developer", "skills": ["SQL", "Python"]}');
```

Stores JSON Data in SQL.

```
-- Method 1
SELECT name, details->>'$.skills[0]' AS first_skill FROM Employees;

-- Method 2
SELECT name, JSON_EXTRACT(details, '$.skills[0]') AS first_skill FROM Employees;
```

Extracts JSON Values (MySQL Example)

name	first_skill
Alice	SQL

```
UPDATE Employees
SET details = JSON_SET(details, '$.position', 'Senior Developer')
WHERE id = 1;
```

Updates the position field inside JSON.

```
SELECT * FROM Employees
WHERE JSON_CONTAINS(details, '"Python"', '$.skills');
```

Finds employees whose skills include "Python".

Some databases (SQL Server, PostgreSQL, MySQL) provide native XML support. You can store XML data in an XML column type. SQL provides functions to parse, query, and update XML data.

```
CREATE TABLE Products (
  id INT PRIMARY KEY,
  name VARCHAR(50),
  specs XML -- Stores structured XML data
);

INSERT INTO Products (id, name, specs)
VALUES (1, 'Laptop', 'Dell1000');
```

Stores XML Data.

```
SELECT name, specs.value('/product/brand')[1], 'VARCHAR(50)' AS brand
FROM Products;
```

Extracts values in SQL Server.

name	brand
Laptop	Dell

```
UPDATE Products
SET specs.modify('replace value of (/product/price)[1] with 1200')
WHERE id = 1;
```

Updates inside XML.

```
SELECT * FROM Products
WHERE specs.exist('/product[brand="Dell"]') = 1;
```

Finds products where is "Dell".

CASE

CASE lets you return different values based on conditions. It's used in SELECT, WHERE, ORDER BY, and even in UPDATE statements.

```
CASE expression
  WHEN value1 THEN result1
  WHEN value2 THEN result2
  ...
  ELSE default_result
END
```

Simple CASE.

```
CASE
  WHEN condition1 THEN result1
  WHEN condition2 THEN result2
  ...
```

```
ELSE default_result
END
```

Searched CASE.

```
SELECT name, score,
CASE
    WHEN score >= 90 THEN 'A'
    WHEN score >= 80 THEN 'B'
    WHEN score >= 70 THEN 'C'
    WHEN score >= 60 THEN 'D'
    ELSE 'F'
END AS grade
FROM Students;
```

Returns:

name	score	grade
Alice	95	A
Bob	82	B
Carol	68	D
Dave	55	F

```
SELECT name, score
FROM Students
ORDER BY
CASE
    WHEN score < 60 THEN 1
    ELSE 2
END;
```

In this example we use CASE in ORDER BY. It sorts students based on pass/fail, prioritizing those who failed.

```
UPDATE Students
SET grade =
CASE
    WHEN score >= 90 THEN 'A'
```

```
WHEN score >= 80 THEN 'B'
WHEN score >= 70 THEN 'C'
WHEN score >= 60 THEN 'D'
ELSE 'F'
END;
```

This example shows using CASE in UPDATE.

Operators

Operator	Description	Example
=	Equal to	salary = 5000
!= or <>	Not equal to	name <> 'Bob'
>	Greater than	age > 30
<	Less than	price < 100
>=	Greater than or equal	score >= 80
<=	Less than or equal	rank <= 5

Comparison operators (used to compare two values).

Operator	Description	Example
AND	All conditions must be true	age > 25 AND city = 'Paris'
OR	At least one condition is true	age < 18 OR age > 60
NOT	Negates a condition	NOT (status = 'inactive')

Logical operators (used to combine multiple conditions).

Operator	Description	Example
+	Addition	price + tax
-	Subtraction	salary - bonus
*	Multiplication	quantity * rate
/	Division	total / count
%	Modulus (remainder)	score % 2

Arithmetic operators (used to perform calculations).

Operator	Description	Example
----------	-------------	---------

Operator	Description	Example
BETWEEN	Value within a range	age BETWEEN 18 AND 30
IN	Value matches any value in a list	country IN ('US', 'UK', 'IN')
LIKE	Pattern matching using % and _	name LIKE 'A%' (starts with A)
IS NULL	Checks if value is null	email IS NULL
IS NOT NULL	Checks if value is not null	email IS NOT NULL

BETWEEN, IN, LIKE, IS NULL (special comparison operators).

Operator	Description
&	Bitwise AND
,	,
^	Bitwise XOR
~	Bitwise NOT

Bitwise operators (less common, used for bit-level comparisons).

```
SELECT name, salary
FROM Employees
WHERE department = 'Sales'
      AND salary BETWEEN 4000 AND 6000
      AND name LIKE 'A%';
```

This example returns employees in Sales with salary between 4000 and 6000, whose names start with "A".

LIKE

The LIKE operator is used in a WHERE clause to search for a specified pattern in a column (usually a VARCHAR, TEXT, or CHAR type).

- % - Matches zero or more characters
- _ - Matches exactly one character
- MySQL - Not case-sensitive by default (unless using a binary collation).
- PostgreSQL - Case-sensitive unless you use ILIKE (case-insensitive LIKE).
- SQL Server - Depends on the collation settings.

```
SELECT column1, column2, ...
```

```
FROM table_name  
WHERE column_name LIKE pattern;
```

Basic syntax using LIKE.

```
SELECT * FROM Customers  
WHERE name LIKE 'A%';
```

Finds all names that start with 'A' (e.g., Alice, Andrew, Anna).

```
SELECT * FROM Customers  
WHERE name LIKE '%son';
```

Finds names ending with 'son' (e.g., Jackson, Samson).

```
SELECT * FROM Products  
WHERE description LIKE '%organic%';
```

Finds descriptions containing 'organic' anywhere.

```
SELECT * FROM Users  
WHERE username LIKE 'm_ke';
```

Matches names like mike, make, etc. (any 4-letter word starting with "m" and ending with "ke").

```
SELECT * FROM Books  
WHERE title NOT LIKE 'The%';
```

Excludes all titles that start with "The".

IN



The IN operator allows you to test whether a value exists in a set of values (like a list or the result of a subquery).

```
SELECT column1, column2, ...  
FROM table_name  
WHERE column_name IN (value1, value2, value3, ...);
```

Basic syntax.

```
SELECT name, department
FROM Employees
WHERE department IN ('HR', 'IT', 'Marketing');
```

-- It's the same as:

```
WHERE department = 'HR' OR department = 'IT' OR department = 'Marketing'
```

This is an example of IN with a list of values. It returns employees whose department is HR, IT, or Marketing.

```
SELECT name
FROM Students
WHERE class_id IN (
    SELECT id
    FROM Classes
    WHERE teacher = 'Ms. Smith'
);
```

This is an example of IN with a subquery. This returns students who are in classes taught by Ms. Smith.

```
SELECT * FROM Products
WHERE category NOT IN ('Clothing', 'Electronics');
```

Example of using NOT IN. This returns products not in those categories. Be careful when using NOT IN with NULL values — it can behave unexpectedly if the subquery returns a NULL.

Aliases

An alias is a temporary name given to a table or column in a query. It doesn't change the actual name in the database — it just gives you a short or descriptive name to work with within the context of that query.

```
-- Method 1
SELECT column_name AS alias_name
FROM table_name;

-- Method 2
```

```
SELECT column_name alias_name
FROM table_name;
```

Example of using column aliases.

```
SELECT t.column_name
FROM table_name AS t;
```

Table alias.

```
SELECT salary AS "Monthly Salary"
FROM Employees;
```

If your alias has spaces or special characters, wrap it in double quotes or square brackets depending on the database.

```
SELECT price * quantity AS total_cost
FROM Orders;
```

In this example the result will have a nicely labeled column: total_cost.

Triggers

A trigger is a special kind of stored procedure that automatically runs when a specific event occurs on a table or view.

Triggers are fired automatically in response to these events:

- INSERT
- UPDATE
- DELETE

They can be executed:

- BEFORE the event happens (to validate or modify)
- AFTER the event happens (to log, audit, or react)

```
CREATE TRIGGER log_insert
AFTER INSERT ON Employees
FOR EACH ROW
BEGIN
```



```
INSERT INTO LogTable (event_type, employee_id, action_time)
VALUES ('INSERT', NEW.id, CURRENT_TIMESTAMP);
END;
```

Creates a trigger. In this example we log every insert happening done to our Employees table to a table called log_insert.

```
CREATE TRIGGER validate_salary
BEFORE UPDATE ON Employees
FOR EACH ROW
BEGIN
    IF NEW.salary < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Salary cannot be negative';
    END IF;
END;
```

BEFORE UPDATE trigger. In this example the trigger blocks updates that try to set a salary below 0.

The keywords inside triggers:

- NEW: Refers to the new row values (for INSERT or UPDATE)
- OLD: Refers to the old row values (for UPDATE or DELETE)

N+1 problem

The N+1 problem happens when a code or query ends up executing 1 query to get a list of records, and then 1 additional query for each of those records to fetch related data. So instead of 1 efficient query with a JOIN, it's 1 query to get N rows, and then N queries, 1 for each row, to fetch related data. This means a total of N + 1 queries, hence the name.

It's very inefficient, especially as N grows, because it can kill performance, leading to hundreds or thousands of unnecessary database calls.

```
-- Step 1: Get all authors
SELECT * FROM Authors;

-- Step 2: For each author, get their books
SELECT * FROM Books WHERE AuthorID = 1;
```

```
SELECT * FROM Books WHERE AuthorID = 2;
-- And so on...
```

This is an example of the N+1 problem. If there are a 100 authors, this results in $1 + 100 = 101$ queries.

```
SELECT Authors.Name, Books.Title
FROM Authors
JOIN Books ON Authors.AuthorID = Books.AuthorID;
```

This is a solution to the N+1 problem, as it is just 1 query that gets all the data at once.

Database Metadata / System Information

```
-- MySQL:
SELECT @@VERSION;
```

```
-- PostgreSQL:
SELECT version();
```

Shows SQL Server version and OS info.

E.g., 10.5.29-MariaDB-deb11-log

```
-- MySQL:
SELECT DATABASE(); -- Current database
SELECT USER(); -- Current user
SELECT CURRENT_USER(); -- Connection info
```

```
-- PostgreSQL:
SELECT current_database(); -- Current database
SELECT current_user; -- Current user
SELECT session_user; -- Connection info
```

Gives information about current database, user, connection.

```
-- MySQL:
SHOW DATABASES; -- List all databases
```

```
SHOW CREATE DATABASE db_name; -- Specific database info
```

```
-- PostgreSQL:
```

```
SELECT datname FROM pg_database; -- List all databases
```

```
SELECT * FROM pg_database WHERE datname='db_name'; -- Specific database info
```

Shows information about databases.

```
-- MySQL:
```

```
SELECT * FROM INFORMATION_SCHEMA.TABLES WHERE TABLE_SCHEMA='db_name'; -- List tables
```

```
SELECT * FROM INFORMATION_SCHEMA.COLUMNS WHERE TABLE_NAME='table_name'; -- List columns
```

```
SELECT * FROM INFORMATION_SCHEMA.TABLE_CONSTRAINTS WHERE TABLE_NAME='table_name'; -- List constraints
```

```
-- PostgreSQL:
```

```
SELECT * FROM INFORMATION_SCHEMA.TABLES WHERE table_schema='public'; -- List tables
```

```
SELECT * FROM INFORMATION_SCHEMA.COLUMNS WHERE table_name='table_name'; -- List columns
```

```
SELECT * FROM INFORMATION_SCHEMA.TABLE_CONSTRAINTS WHERE table_name='table_name'; -- List constraints
```

Shows information about tables.

```
-- MySQL:
```

```
SHOW VARIABLES; -- General server variables
```

```
SHOW max_connections; -- Specific variable
```

```
-- PostgreSQL:
```

```
SHOW ALL; -- General server variables
```

```
SHOW max_connections; -- Specific variable
```

Shows information about server configuration.

This document's home with other cheat sheets you might be interested in:

<https://gitlab.com/davidvarga/it-cheat-sheets>

Accessibility touches by:

Mr. Double D

Sources:

<https://www.wikipedia.org/>

<https://stackoverflow.com/>

<https://dev.to/>

<https://www.w3schools.com/>

License:

GNU General Public License v3.0 or later